Assignment 1 (Package – Collections & Exception Handling)

1. Book Management using ArrayList

Problem Statement

Develop a **menu-driven Book Management System** using an `ArrayList<Book>`.

Create a `Book` class with the following attributes:

- `bookId`
- `title`
- `author`
- `price`

Display the following menu:

```
1. Add Book
2. Display All Books
3. Search Book by ID
4. Update Book
5. Delete Book
6. Exit
```

The program should allow the user to perform the selected operation and continue displaying the menu until **Exit** is selected.

Use appropriate `ArrayList` methods to perform the operations.

Concepts: `ArrayList`, Objects, `add()`, `get()`, `set()`, `remove()`, `size()`, iteration.

---

2. Book Management using HashMap

Problem Statement

Develop a **menu-driven Book Management System** using a `HashMap<Integer, Book>`.

Create a `Book` class with:

- `bookId`
- `title`
- `author`
- `price`

Use the **book ID as the key** and the **Book object as the value**.

Display the following menu:

```
1. Add Book
2. Display All Books
3. Search Book by ID
4. Update Book
5. Delete Book
6. Exit
```

The program should allow the user to perform the selected operation and continue until **Exit** is selected.

Use appropriate `HashMap` methods to perform the operations.

Concepts: `HashMap`, Objects, `put()`, `get()`, `putIfAbsent()`, `remove()`, `containsKey()`, `entrySet()`.

---

3. Stack – Browser History

Develop a program to simulate **Browser History** using a `Stack`.

Operations:

```
1. Visit New Page
2. Go Back
3. View Current Page
4. Display History
5. Exit
```

The most recently visited page should be removed first when the user selects **Go Back**.

Concepts: `push()`, `pop()`, `peek()`, `isEmpty()`.

---

4. LinkedList – Train Passenger Management

Develop a program to manage train passengers using a `LinkedList`.

Perform operations such as:

```
1. Add Passenger at Beginning
2. Add Passenger at End
3. Remove First Passenger
4. Remove Last Passenger
5. Search Passenger
6. Display Passengers
7. Exit
```

Concepts: `LinkedList`, `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`.

---

5. Queue – Hospital Patient Management

Develop a **Hospital Patient Management System** using a `Queue`.

Patients should be treated according to **FIFO (First In, First Out)**.

Operations:

```
1. Add Patient
2. Treat Patient
3. View Next Patient
4. Display Waiting Patients
5. Exit
```

Concepts: `offer()`, `poll()`, `peek()`, `isEmpty()`.

---

6. Exception Handling – Bank Withdrawal

Develop a Java program for a **Bank Account Withdrawal System**.

Create an `Account` class with account number, account holder name, and balance.

Implement a `withdraw()` method that:

- Checks whether the withdrawal amount is positive.
- Checks whether sufficient balance is available.
- Uses `throw` when the withdrawal is invalid.

Handle the exception using:

```
try
catch
finally
throw
```

---

7. Custom Exception – Age Validation

Develop a Java program to validate a person's age for **event registration**.

Create a custom exception named:

```
InvalidAgeException
```

If the age entered is below 18, throw the custom exception with an appropriate message.

Use:

```
try
catch
throw
throws
```

to handle the exception.